

Síntesis de katas, mock-parcial integrador y preguntas MC sobre estilos arquitectónicos

Material de estudio derivado del corpus de la cátedra: ADD e-commerce (Clase 3), Control de fábrica con sensores serie (Clase 4), Healthcare.gov (Clase 4, ATAM post-mortem), Twitter Big Data en tiempo real (Clase 6) y Compraventa de Acciones (Clase 7).

Parte 1 — Síntesis y mock-parcial

1.1. Esqueleto común a las 5 katas de clase

Las 5 katas siguen el mismo flujo procedimental, con variaciones por dominio.

Paso	Qué se hace	Origen en el corpus
1. Stakeholders + sistemas externos	Actores humanos, BCs externos, reguladores y sus preocupaciones.	Clase 7 «1ª iteración»; Ejercicio 2013 (a).
2. Candidatos → ranking de atributos	Enumerar atributos ISO 25000; justificar el #1 por la naturaleza de los datos / del dominio .	Clase 7: «la importancia se determina por la naturaleza de los datos».
3. Árbol de utilidad con escenarios (Imp, Dif)	Cada atributo se aterriza en escenarios <i>medibles</i> .	Ejercicio 2013 (b); Healthcare.gov post-hoc.
4. Iteración por atributo	Estructura base → mecanismos del #1 → #2 sin romper #1 → ...	Clase 7: «primero el problema principal, después los accesorios».
5. Decisiones con trade-offs explícitos + CAP	Tabla Decisión / Opciones / Elección / Justificación + posición CP/AP.	Las 5 katas.
6. Validación con Lightweight ATAM	Sensitivity points, tradeoff points, riesgos.	Healthcare.gov es esto en modo post-mortem.

Heurísticas que se repiten (las «reglas no escritas»)

- La naturaleza del dato manda sobre la intuición de UX.
- *Threshold defense*: subir el costo del atacante por encima de su retorno esperado.
- Persistencia poliglota cuando hay sub-sistemas con perfiles divergentes.
- Segregación física por *blast radius* (Ops en cluster aparte vía VPN).
- Productor-Consumidor cuando hay asimetría de velocidad.
- Redundancia con heartbeat en componentes críticos del path productivo.
- Streaming = WebSocket; transacción discreta = Request-Response.
- Edge canónico = WAF + LB-reverse-proxy + HTTPS end-to-end + MFA en login.
- CAP siempre declarado, posiblemente distinto por sub-sistema.

Anti-patrones del corpus (Healthcare.gov)

Saltar a cajas antes de articular atributos; scope creciendo bajo plazo político inmovible; integraciones tardías sin contrato; un solo motor para todo; sin queue absorber entre front y core; BDUF sin la rigurosidad que BDUF exige.

1.2. Mock — Enunciado (estilo parcial)

Ingeniería de Software II — Parcial práctico (caso integrador)

Caso: Sistema de Cobro Electrónico de Peajes — *TelePeaje*

Una concesionaria de autopistas nacionales debe reemplazar el sistema de cobro manual por un **sistema de cobro electrónico** que opere en **180 cabinas distribuidas en 12 corredores viales**. Cada cabina cuenta con:

- Un **lector RFID** del transponder del vehículo (cuenta prepaga del cliente).
- Una **cámara LPR** (Licence Plate Recognition) que fotografía la patente como respaldo y para infracciones.
- Una **barrera automática** con sensor de paso.
- Un **gabinete industrial** (PC reforzada) que orquesta los tres anteriores vía puertos serie y Ethernet.

El sistema debe **descontar el peaje del saldo del cliente** en el momento del paso, **levantar la barrera en < 600 ms** desde la lectura del RFID, y **registrar el evento** para liquidación con el concesionario y rendición ante **Vialidad Nacional**. Cuando un vehículo no tiene transponder o saldo insuficiente, debe **emitir una infracción** que se cruza con la base de **AFIP** y el **Registro Automotor**.

El cliente recarga su cuenta a través de **app móvil**, **home banking** (red bancaria), **agentes de pago** (Pago Fácil, Rapipago) y **débito automático**. Las recargas deben verse reflejadas en la cabina en **< 30 s**.

Volumen esperado: ~6 millones de pasos diarios; picos de 12.000 pasos/min en fines de semana largos; ~3 millones de clientes activos.

Restricciones legales: auditoría completa de Vialidad Nacional, conservación de evidencia (foto LPR + ticket digital) por **10 años**, no-repudio del cobro, retención de PII según Ley 25.326. Los datos de los corredores deben residir en **infraestructura local** (no se admite cloud extranjero para datos operativos).

Plazo político: el contrato de la nueva concesión arranca el 1° de enero del próximo año. La fecha es inmovible.

Incisos a resolver

- Identifique stakeholders y sistemas externos. Indique para cada uno sus preocupaciones dominantes.
- Enumere candidatos de atributos de calidad. Justifique un **ranking** de los 4–5 dominantes y declare la posición CAP del sistema (o de cada sub-sistema, si difiere).
- Construya un **árbol de utilidad** con al menos 6 escenarios hoja con (Importancia, Dificultad).
- Aplique **Attribute Driven Design** en iteraciones sucesivas por atributo. Diagrame la arquitectura final con sus componentes y conexiones, justificando cada decisión.
- Para cada atributo trabajado, indique al menos un **trade-off explícito**.
- Realice una pasada de **Lightweight ATAM** identificando 3 sensitivity points, 2 tradeoff points y los 3 riesgos principales con su mitigación.

Duración: 2 h 30 min. Hoja entregada: una. Las decisiones sin justificación no puntúan.

1.3. Mock — Resolución modelo

(a) Stakeholders y sistemas externos

Stakeholder / sistema	Preocupación dominante
Conductor (cliente) con transponder	Que la barrera se levante rápido y se le descuente lo correcto.
Conductor sin transponder	Que la infracción esté correctamente notificada y sea apelable.
Operador de cabina	Que la consola interna funcione y le avise de excepciones.
Concesionaria	Liquidación correcta; mínimo downtime — un corredor caído pierde dinero por minuto.
Vialidad Nacional	Auditability — evidencia disponible 10 años, trazabilidad bit-a-bit.
AFIP / Registro Automotor	Interfaz estable para cruces de infracciones.
Red bancaria / agentes de pago	Contrato de integración (settlement diario).
Ciudadanía / prensa	Que no haya escándalo de cobros indebidos (riesgo Healthcare.gov).
Ops del concesionario	Consola interna, kill-switch de cabina, dashboards de salud de flota.

Sistemas externos: AFIP, Registro Automotor, red bancaria (Banelco/Link), Pago Fácil, Rapipago, SMTP, sistema de Vialidad Nacional.

(b) Atributos candidatos y ranking

Candidatos: **Availability, Auditability, Performance, Reliability, Security, Interoperability, Scalability, Usability.**

Ranking justificado:

1. **Availability** — una cabina caída forma cola física. El sistema *vive en una autopista*: el costo de downtime es operacional y reputacional inmediato.
2. **Reliability + Auditability** — Vialidad Nacional exige evidencia inmutable y no-repudio. Sin auditoría válida, el cobro es impugnabile.
3. **Performance** — 600 ms de la lectura al levantamiento de barrera es un *hard real-time* acotado.
4. **Security** — datos de pago y PII bajo Ley 25.326. *Threshold defense*.
5. **Interoperability** — múltiples redes de recarga + AFIP + Registro + Vialidad, cada una con su protocolo.
6. **Scalability** — picos de fin de semana largo predecibles → elasticidad.

Posición CAP — distinta por sub-sistema (lección Twitter):

- **Cabina ↔ saldo del cliente → AP con conciliación posterior.** La cabina no puede esperar consenso global de 30 ms para levantar la barrera; debe operar con una caché local del saldo y reconciliar contra central de manera asincrónica.
- **Liquidación / auditoría / facturación → CP.** El estado contable consolidado no admite divergencia.

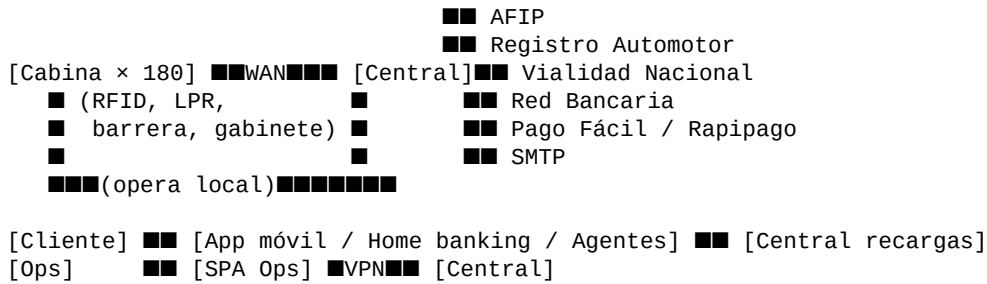
(c) Árbol de utilidad — escenarios hoja

Atributo	Escenario	(Imp, Dif)
Availability	Caída del enlace WAN del corredor: la cabina sigue cobrando ≥ 4 h contra caché local sin perder eventos.	(H, H)
Availability	Caída de un gabinete: failover a su par dentro de la misma cabina sin dejar la barrera abierta.	(H, H)
Performance	p99 lectura RFID → barrera arriba < 600 ms.	(H, M)

Atributo	Escenario	(Imp, Dif)
Auditability	Vialidad solicita reconstruir el cobro de un vehículo de hace 7 años: foto LPR + ticket + saldo antes/después en < 1 min.	(H, M)
Security	Robo del transponder no permite consumir saldo sin notificar al cliente (lockable + MFA al recargar).	(H, M)
Interoperability	Caída de AFIP no detiene cobros, sólo demora el cruce de infracciones.	(H, L)
Scalability	12.000 pasos/min en feriado largo sin degradar p99.	(M, H)
Reliability	Recarga vista por la cabina en < 30 s para el 99% de las recargas.	(H, M)

(d) ADD iterativo

1ª iteración — Estructura base (sistemas externos y funcionales).



2ª iteración — Availability (#1). Cada cabina con gabinete redundante activo-pasivo con *heartbeat* (lección directa Ejercicio 2013: pollers redundantes). Caché local del saldo (modelo offline-first de PDV electrodomésticos). Buffer de eventos persistente local en cada cabina: si la WAN cae, los eventos se almacenan y reenvían con *idempotency_key* (lección Twitter social graphs: *write operations are idempotent, retry until success*). Central en arquitectura activo-activo multi-DC dentro del país (data residency obligatoria). Circuit breakers en cada integración externa: caída de AFIP no detiene cobros.

3ª iteración — Reliability + Auditability (#2). Event sourcing del evento «paso» sobre log append-only firmado: (timestamp, cabina_id, transponder_id, patente_LPR_hash, monto, saldo_antes, saldo_después) firmado por HSM de la cabina y contrafirmado al llegar a central. La foto LPR se guarda en almacenamiento WORM con hash incluido en el evento. La proyección contable y los reportes se derivan del log; el log es la fuente de verdad. Conservación 10 años en frío con replicación geográfica.

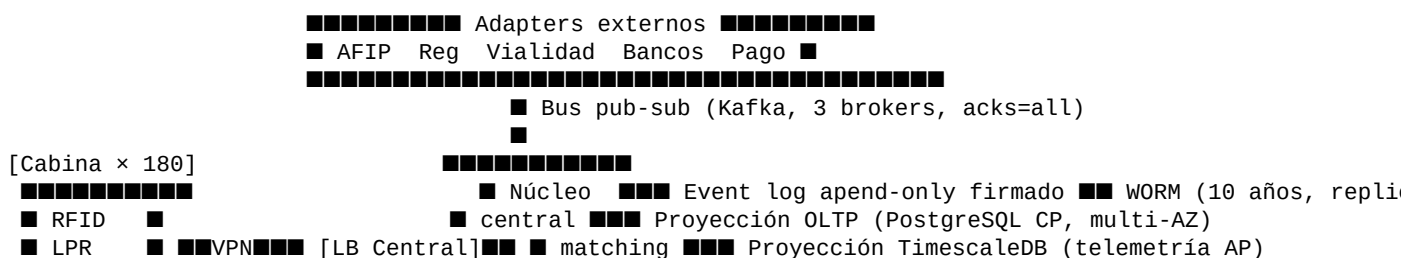
4ª iteración — Performance (#3). La cabina **no consulta central en el camino crítico**. Mantiene caché local LRU del saldo de los transponders frecuentes del corredor (principio de localidad — lección Twitter). El gabinete es embedded con software determinístico, sin GC tóxico en el path crítico. Patrón Productor-Consumidor: el lector RFID es el productor, el control de barrera consume en orden FIFO con cola in-memory bounded.

5ª iteración — Security (#4). WAF + LB delante de la app móvil y del backoffice; MFA para recargas; HTTPS end-to-end; encriptación at-rest del log y de la base operativa; VPN entre cabinas y central (toda comunicación WAN sobre VPN sitio-a-sitio, lección Compraventa). Cluster Ops segregado vía VPN (blast radius). HSM por cabina y por DC central. Tokenización del PAN bancario en recargas (no almacenar PAN claro, lección PCI).

6ª iteración — Interoperability (#5). Patrón adapter por integración (lección Control de fábrica): un adapter por cada red de recarga, uno para AFIP, uno para Registro, uno para Vialidad. Cada adapter traduce el protocolo idiosincrásico a un evento canónico interno que viaja por bus pub-sub. Cuando aparezca un nuevo agente de pago, se agrega un adapter sin tocar el core.

7ª iteración — Scalability (#6). Sharding por *corredor* (cada corredor es embarrassingly parallel respecto de los demás); la consolidación nacional es batch nocturno. Time-series DB (TimescaleDB) para telemetría de salud de cabinas y dashboards, separada del OLTP transaccional (lección persistencia poliglota Twitter). CDN para assets de la app y del portal público.

Diagrama final (resumen):



```
■ Barrera ■ WAF + LB ■ saldos ■■■ Proyección Memcached (saldo caliente)
■ Gab. ■ (Activo-Activo) ■■■■■■■■■■
■ caché ■ ■
■ saldo ■ ■■■■ [Cluster Ops] (VPN, segregado)
■ buffer ■ ■■■■ [App móvil + Home banking + Agentes]
■ HSM ■
■ Heart- ■
■ beat ■ Pollers redundantes activo-pasivo
■■■■■■■■■■
```

(e) Trade-offs explícitos por atributo

Atributo	Trade-off pagado
Availability	Caché local del saldo viola consistencia fuerte (AP en path crítico). Aceptado por real-time físico; reconciliación post-hoc cierra la cuenta.
Auditability ↔ Performance	Firma HSM por evento agrega ~3–5 ms. Aceptable porque el SLA es 600 ms total; el grueso es lectura RFID + mecánica.
Reliability ↔ Simplicidad	Gabinete redundante con heartbeat duplica hardware y agrega coordinación de leader. Justificado por costo de downtime.
Security ↔ Usability	MFA en recargas agrega fricción. Aceptable porque la frecuencia de recarga es baja (1× mes promedio).
Interoperability ↔ Acoplamiento de release	Adapter por integración exige versionado del evento canónico. Es el precio del desacoplamiento.
Scalability ↔ Consistencia global	Sharding por corredor exige proceso nocturno de consolidación. La contabilidad consolidada queda en T+1, no en tiempo real.

(f) Lightweight ATAM

Sensitivity points

1. **El evento «paso» firmado por HSM y replicado al WORM.** Si la firma o la replicación fallan, no hay cobro válido. Toda la Auditability depende de este componente.
2. **La caché local de saldo en la cabina.** Si su consistencia se degrada (TTL mal calibrado, sync lento), aparecen cobros indebidos o pasos sin cobrar. Es el componente más sensible al *clock skew* y al *split-brain* central-cabina.
3. **El reloj del gabinete.** El timestamp del evento es prueba legal. NTP de grado industrial + monotonic clock; deriva > 100 ms debe disparar alarma.

Tradeoff points

1. **AP vs CP del saldo.** Decisión central del caso. Resuelto a favor de AP local + reconciliación, contra la intuición «es plata, debe ser CP».
2. **Plazo político (1° enero) vs scope completo.** La fecha es inamovible. Se exige negociar MVP por corredor (lección Healthcare.gov #4 Simplification): habilitar los 12 corredores de a uno, no big-bang.

Riesgos principales y mitigaciones

Riesgo	Mitigación
Plazo político y scope creciendo (riesgo Healthcare.gov #2 + #4).	MVP por corredor + Architectural Guardrails firmados por sponsor + freeze de features 30 días antes.
Caída de la WAN sostenida en un corredor remoto.	Cabina cobra en modo offline ≥ 4 h sobre caché + buffer; reconciliación al volver el enlace.
Cobro indebido (split-brain saldo).	Política de <i>conservadurismo a favor del cliente</i> : ante divergencia detectada, se acredita y se notifica; auditoría humana para casos > umbral.

Pregunta a profundizar

¿Cómo cambia la arquitectura si Vialidad Nacional exige que el log inmutable corra en su propia infraestructura (no del concesionario)? ¿Es viable un esquema de notarización con blockchain permitida interagencial, o introduce más complejidad de la que resuelve? (Pista: Lamport — Paxos; Castro & Liskov — PBFT.)

Parte 2 — Multiple choice sobre los 7 estilos

Una pregunta por estilo. Cada una: contexto corto, 4 opciones, una correcta, justificación con referencia al catálogo de Estilos arquitectónicos y a POSA cuando aplica.

Pregunta 1 — Mails automáticos a fin de mes

Contexto. Un sistema contable debe enviar el resumen mensual a 250.000 clientes el último día de cada mes a las 03:00. Cada resumen requiere: consolidar movimientos, generar PDF, firmar digitalmente, archivar y enviar por SMTP. No hay urgencia de tiempo real, pero todo debe terminar antes de las 06:00. ¿Qué estilo arquitectónico encaja mejor?

- a) Publish-Subscribe — un evento «fin de mes» se publica y cada cliente se procesa por un suscriptor.
- b) **Batch Sequential** — un job nocturno con etapas: consolidar → renderizar PDF → firmar → archivar → enviar.
- c) Client-Dispatcher-Server — el dispatcher rutea cada cliente a un servidor de generación.
- d) Broker — los resúmenes se encolan en un broker y trabajadores los procesan.

Respuesta: b.

Justificación. El problema cumple las dos condiciones canónicas de **Batch Sequential**: (i) el dataset completo se conoce de antemano (todos los clientes del mes), (ii) cada etapa depende de que la anterior termine sobre todo el conjunto, o al menos sobre el lote. No hay ningún driver de tiempo real ni de desacoplamiento. Pub-Sub o Broker son sobre-ingeniería para un proceso con scheduler determinístico y ventana acotada (POSA Vol 1, Buschmann). Si más adelante el volumen exigiera paralelismo intra-etapa, la evolución natural es a **Pipes and Filters** — no a un broker.

Pregunta 2 — Pipeline de procesamiento de imágenes médicas

Contexto. Llegan tomografías que deben pasar por: normalización → segmentación con modelo ML → anotación por radiólogo → compresión DICOM → publicación al HIS del hospital. Cada etapa es un proceso independiente, cada una puede escalarse por separado, y el volumen permite procesar varias imágenes en paralelo en cada etapa. ¿Qué estilo encaja mejor?

- a) Hierarchical Layers — cada etapa es una capa que invoca a la siguiente.
- b) Batch Sequential — todas las imágenes del día en cinco pasadas globales.
- c) **Pipes and Filters** — cada etapa es un filtro que lee de su pipe de entrada y escribe en el de salida.
- d) Client-Dispatcher-Server — el dispatcher elige a qué filtro mandar cada imagen.

Respuesta: c.

Justificación. **Pipes and Filters** es la solución canónica cuando cada unidad de trabajo fluye por una secuencia de transformaciones independientes, los filtros no comparten estado, y se quiere paralelismo por filtro y por elemento (POSA Vol 1, capítulo *Pipes and Filters*). Hierarchical Layers fuerza llamada sincrónica capa-a-capo y mezcla responsabilidades. Batch Sequential pierde el streaming intra-imagen (la imagen N+1 puede entrar al pipeline antes de que la N termine). CDS resuelve routing a réplicas, no orquestación de etapas. Ejemplos canónicos: Unix pipes, Apache NiFi, ffmpeg.

Pregunta 3 — App web tradicional con UI / lógica / datos

Contexto. Una aplicación interna de RR.HH. con interfaz web, lógica de negocio (cálculo de liquidaciones, política de licencias) y persistencia relacional. La empresa quiere que un equipo nuevo pueda mantenerla en 2 años sin perderse, y prioriza claridad sobre features novedosas. ¿Qué estilo encaja mejor?

- a) Publish-Subscribe entre UI, lógica y datos.
- b) **Hierarchical Layers** — Presentación / Aplicación / Dominio / Persistencia, donde cada capa sólo conoce a la inmediata inferior.
- c) Forwarder-Receiver entre UI y backend.
- d) Pipes and Filters con la UI como primer filtro.

Respuesta: b.

Justificación. Cuando el atributo dominante es **maintainability** y el sistema no es distribuido, **Hierarchical Layers** es la elección de menor sorpresa: vocabulario universalmente entendido, dependencia direccional clara, testing por capa, sustitución de capa inferior (cambio de motor de BD) sin tocar las superiores (*Bass Software Architecture in Practice*; Buschmann POSA). Pub-Sub introduce asincronía y complejidad operativa innecesarias para un sistema mono-proceso. Forwarder-Receiver resuelve abstracción de red — no hay red acá. Pipes and Filters no es para CRUD interactivo.

Pregunta 4 — Confirmación de compra que dispara siete acciones

Contexto. Cuando una orden de compra se confirma, deben enterarse: el almacén (reservar stock), facturación, el mailer (mail al cliente), el CRM, el motor antifraude, el BI y el módulo de fidelización. El equipo de Ventas no quiere conocer a esos siete consumidores ni acoplarse a su disponibilidad; mañana puede aparecer un octavo. ¿Qué estilo encaja mejor?

- a) Broker con cola point-to-point — Ventas envía un mensaje y el broker lo entrega a un único trabajador.
- b) **Publish-Subscribe** — Ventas publica OrderConfirmed y cada consumidor se suscribe al tópico.
- c) Client-Dispatcher-Server — el dispatcher elige a qué servicio mandar el evento.
- d) Pipes and Filters — los siete consumidores en serie.

Respuesta: b.

Justificación. El driver dominante es **desacoplamiento de productor de consumidores múltiples y evolutivos**. **Publish-Subscribe** (POSA) lo resuelve exactamente: el productor sólo conoce el tópico, los consumidores se agregan/quitan sin tocarlo, fan-out 1→N nativo. Un Broker genérico también puede hacerlo, pero point-to-point implica un consumidor por mensaje — pierde el 1→N. CDS está pensado para resolver qué servidor atiende a qué cliente (descubrimiento), no para fanout de eventos. Pipes and Filters fuerza acoplamiento secuencial y orden. Distinción fina: Kafka es un broker que ofrece semántica pub-sub. En el examen, «Broker» como estilo refiere al mediador genérico (CORBA-style), no a la implementación concreta.

Pregunta 5 — Múltiples servicios heterogéneos con entrega garantizada

Contexto. Una empresa tiene 35 servicios en distintos lenguajes que deben intercambiar mensajes. Requisitos: el remitente no debe conocer la ubicación física del destinatario, el mensaje debe persistir si el destinatario está caído, debe haber una representación común y una manera estándar de invocar desde cualquiera de los 35 lenguajes. ¿Qué estilo encaja mejor?

- a) Publish-Subscribe puro.
- b) **Broker** — un intermediario que registra servicios, traduce representaciones, persiste y rutea mensajes.
- c) Hierarchical Layers entre los 35 servicios.
- d) Forwarder-Receiver entre cada par.

Respuesta: b.

Justificación. El **Broker** (POSA Vol 1, capítulo *Broker*) es exactamente esto: registro de servicios, transparencia de ubicación, interoperabilidad de lenguaje, persistencia opcional de mensajes. CORBA, RabbitMQ y los message brokers modernos descienden del patrón. Pub-Sub resuelve fanout temático pero no el problema de invocación request-response cross-lenguaje con descubrimiento. Layers no resuelve nada distribuido. Forwarder-Receiver resuelve el cómo de la comunicación entre un par, no el ecosistema de 35 servicios heterogéneos — es complementario al Broker (el Broker internamente puede usar Forwarder-Receiver), no su sustituto.

Pregunta 6 — Abstractar los detalles de red entre dos componentes

Contexto. Tengo dos componentes (un cliente y un servidor de cotizaciones) que deben comunicarse por red. Quiero que el código de negocio del cliente invoque al servidor como si fuera local, sin preocuparse por serialización, protocolo TCP, mapeo de direcciones IP ni gestión de timeouts. No me importa elegir entre múltiples servidores ni hacer fanout — son uno y uno. ¿Qué estilo encaja mejor?

- a) Client-Dispatcher-Server.
- b) Broker.
- c) Forwarder-Receiver.
- d) Publish-Subscribe.

Respuesta: c.

Justificación. **Forwarder-Receiver** (POSA Vol 1) es el patrón de encapsulación de la comunicación entre pares conocidos: el Forwarder del lado del cliente sabe serializar y mandar; el Receiver del lado del servidor sabe recibir y deserializar. Las particularidades del transporte quedan encerradas. Es deliberadamente más liviano que un Broker, justamente porque no hay descubrimiento ni mediación de múltiples participantes. Si después aparece un tercero o se necesita registry → se introduce un Broker; si aparece un pool de servidores equivalentes → CDS. Pero para el problema planteado, Forwarder-Receiver es la elección minimalista correcta.

Pregunta 7 — Pool de servidores equivalentes y selección dinámica

Contexto. Tengo un pool de 8 servidores que ofrecen el mismo servicio de búsqueda full-text. Los clientes no deben conocer las direcciones individuales del pool: alguien debe resolver dinámicamente a cuál instancia rutar cada pedido (por carga, por disponibilidad, por afinidad). Si una instancia cae, el cliente debe seguir funcionando sin enterarse. ¿Qué estilo encaja mejor?

- a) **Client-Dispatcher-Server** — el dispatcher resuelve qué servidor atiende cada pedido y el cliente lo desconoce.
- b) Forwarder-Receiver entre el cliente y cada servidor del pool.
- c) Broker — un mediador completo entre cualquier cliente y cualquier servicio del sistema.
- d) Publish-Subscribe — el cliente publica el query y los servidores compiten por atenderlo.

Respuesta: a.

Justificación. **Client-Dispatcher-Server** (POSA Vol 1) es el patrón canónico de service discovery + transparencia de ubicación dentro de un pool de pares funcionalmente equivalentes. Es el ancestro directo de los service registries modernos (Eureka, Consul) y de los load balancers de capa 7. Forwarder-Receiver no resuelve la selección dinámica — exige conocer el peer. Un Broker es solución de mayor escala (interoperabilidad cross-lenguaje, persistencia, mediación general); para un pool homogéneo es sobre-ingeniería. Pub-Sub competitivo (un consumer group consumiendo) puede emular el efecto, pero pierde la semántica request-response síncrona y la afinidad por cliente.

Tabla resumen — discriminadores rápidos

Si el driver dominante es...	Estilo
Proceso nocturno determinístico, dataset cerrado.	Batch Sequential
Flujo de transformaciones independientes con paralelismo por etapa.	Pipes and Filters
Maintainability de app mono-proceso con UI/lógica/datos.	Hierarchical Layers
Mediación cross-lenguaje con descubrimiento y persistencia.	Broker
Fanout 1→N de eventos a consumidores desacoplados y evolutivos.	Publish-Subscribe
Encapsular transporte entre dos pares conocidos.	Forwarder-Receiver
Selección dinámica dentro de un pool de servidores equivalentes.	Client-Dispatcher-Server